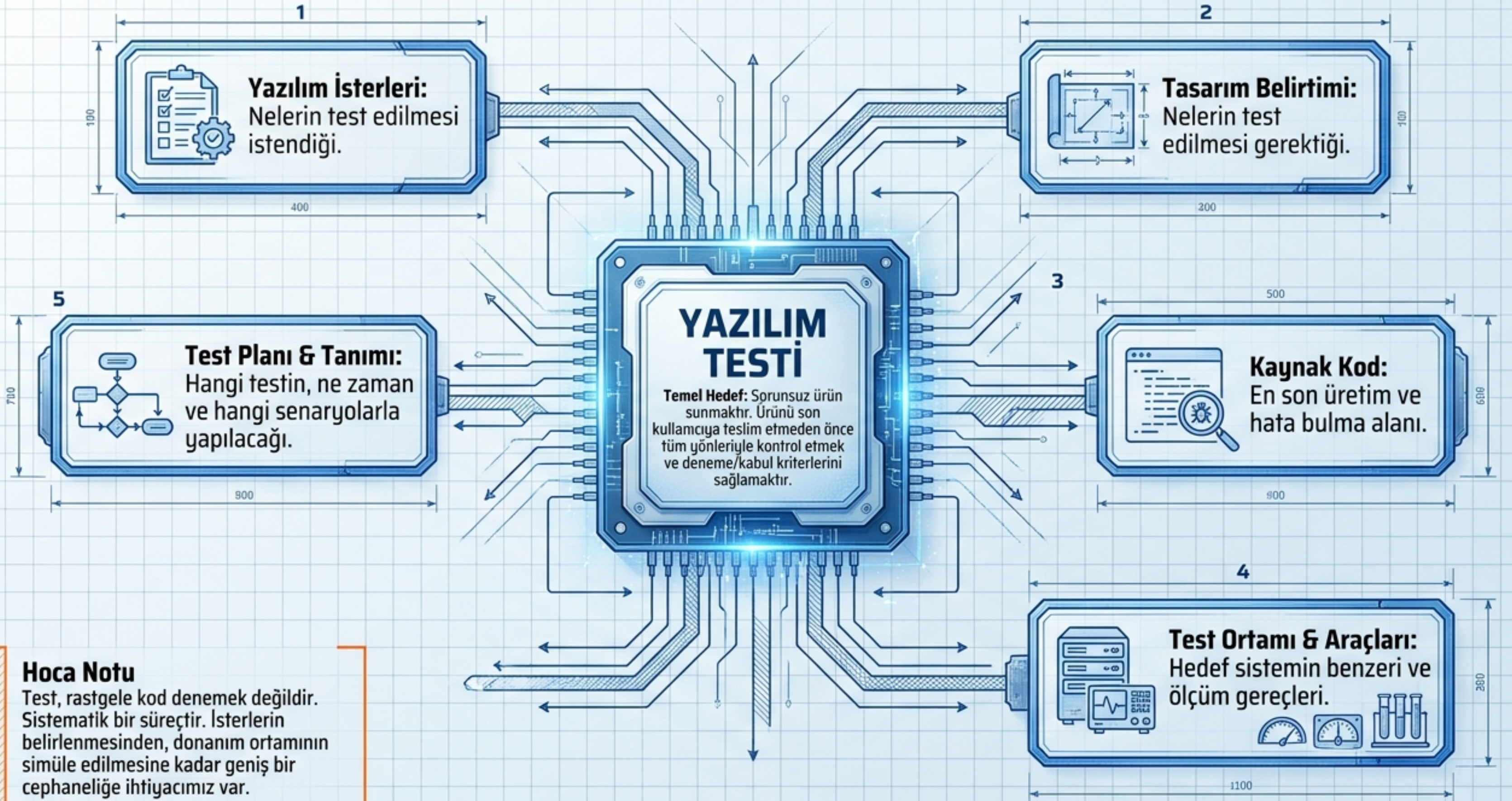


YAZILIM TEST MÜHENDİSLİĞİ

Sorunsuz Ürün Sunma Sanatı
ve Kalite Güvencesi

Hoca Notu: Hoş geldiniz arkadaşlar. Bugün yazılım mühendisliğinin en kritik aşamalarından birini, kodumuzu hayata geçmeden önceki son ve en önemli savunma hattımızı, yani 'Yazılım Testi'ni konuşacağız. Amacımız sadece hata bulmak değil, 'sorunsuz bir ürün' vizyonunu inşa etmektir.

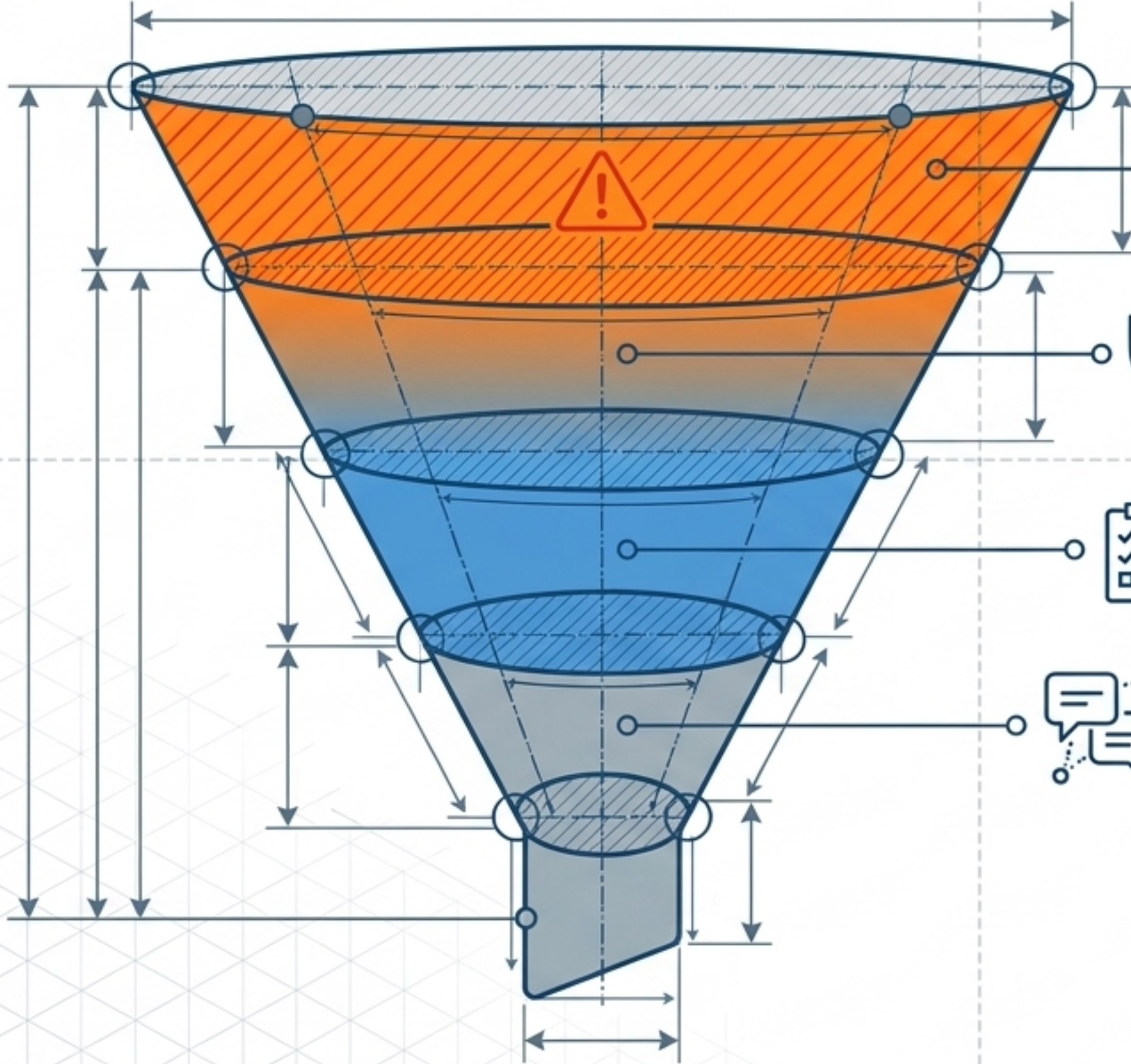


5

Hoca Notu

Test, rastgele kod denemek değildir. Sistematik bir süreçtir. İsterlerin belirlenmesinden, donanım ortamının simüle edilmesine kadar geniş bir cephaneliğe ihtiyacımız var.

BAŞARILI BİR TEST SÜRECİNİN ALTIN KURALLARI



Önce Temeller: Temel fonksiyonları ve son güncellemeleri ilk sıraya alın.



Risk Yönetimi: Genel riskleri, özel risklerden; en sorunlu modülleri, sorunsuz modüllerden önce test edin.



Öncelik Sırası: Beceriye güvenilirlikten önce, müşteri taleplerini ise kendi varsayımlarınızdan önce test edin.

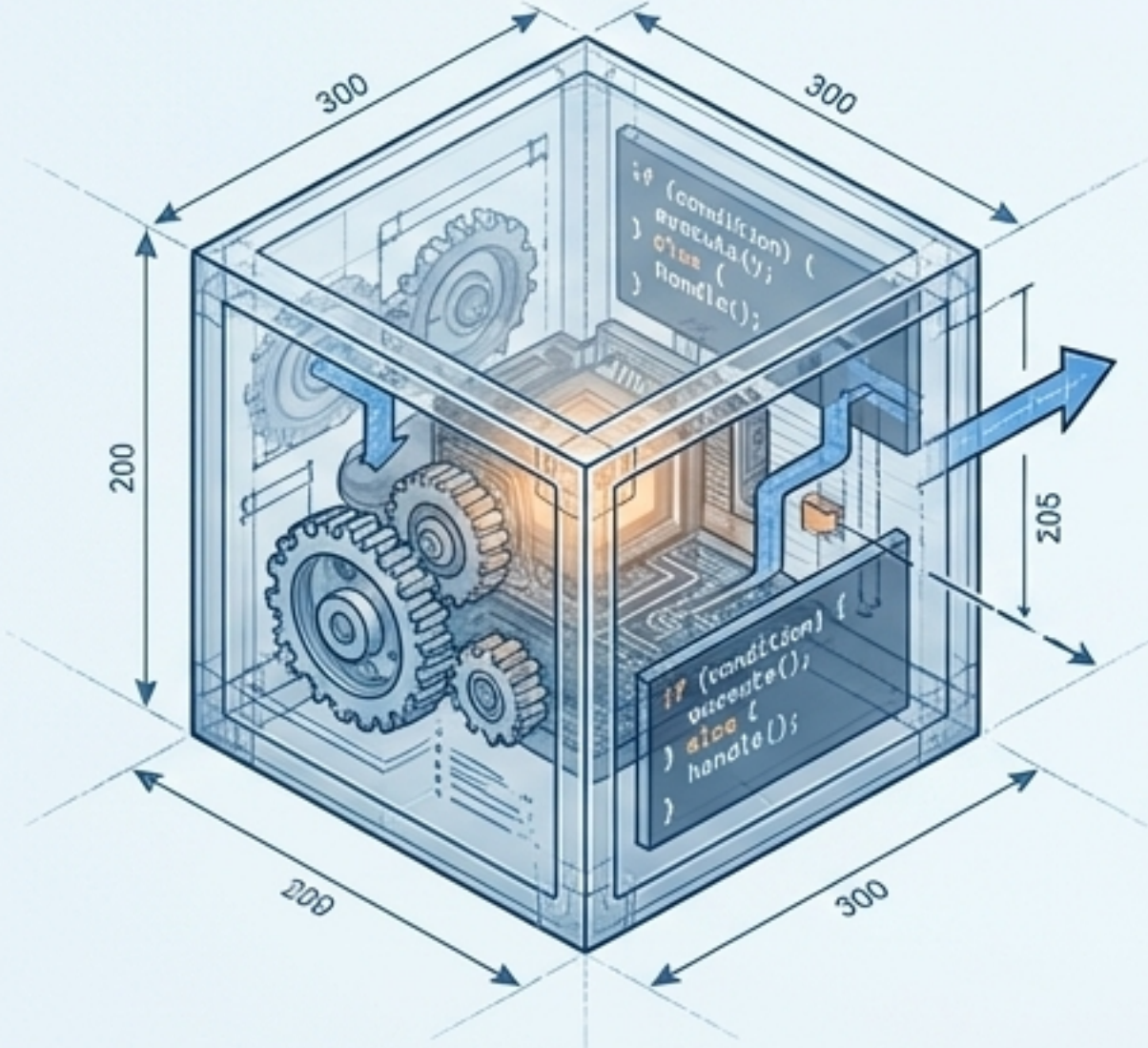


Sürekli İletişim: Her aşamada sonuçları ekiple paylaşın ve önleyici/iyileştirici çalışmalar planlayın.

Hoca Notu: Testte zaman ve kaynak kısıtlıdır. Nereye odaklanacağınızı bilmezseniz, sistem çökerken siz sadece basit bir butonu test ediyordurabilirsiniz. Risk ve etki analizi çok önemlidir.

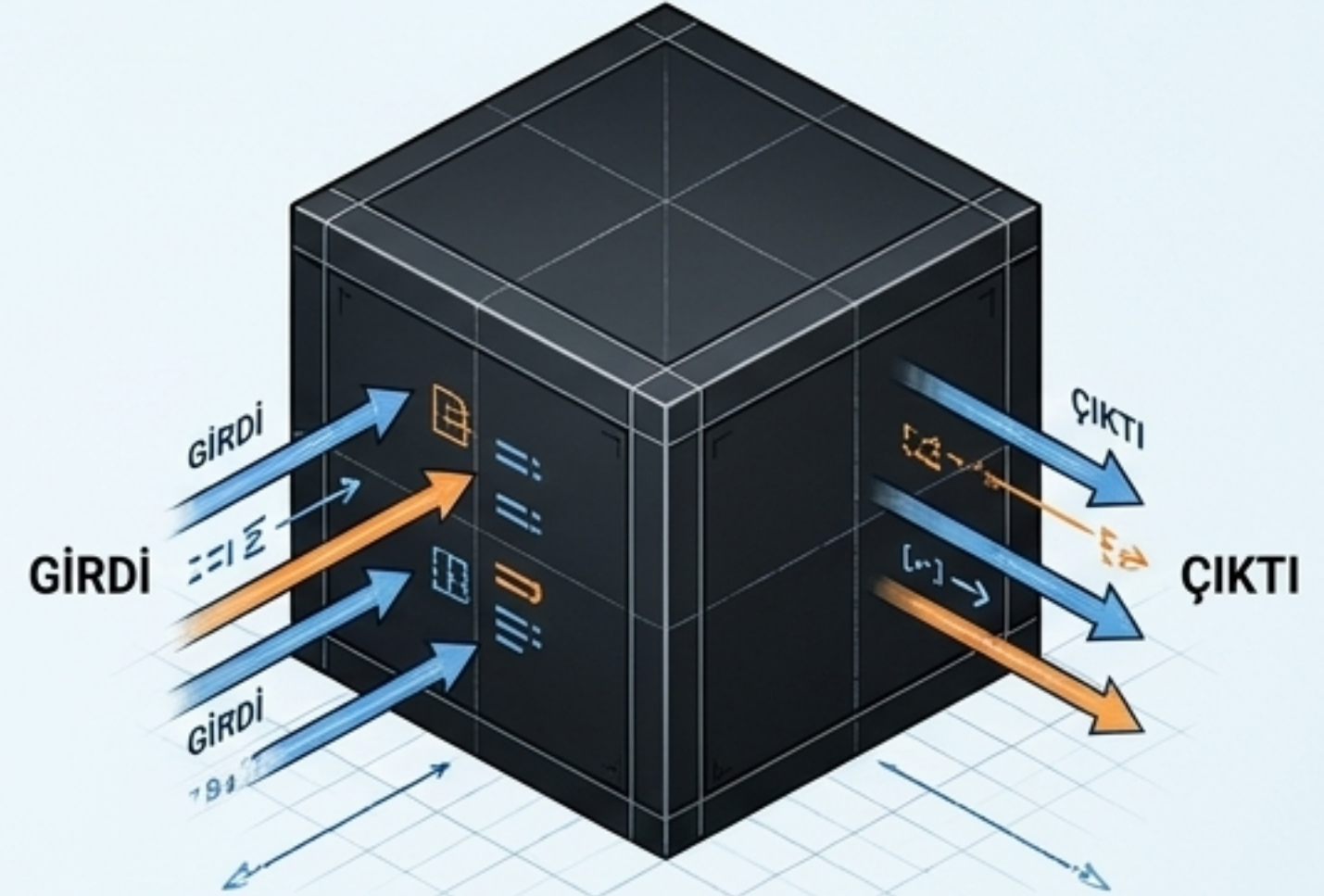
TESTİN İKİ TEMEL PARADİGMASI

Saydam (Beyaz) Kutu Testi



- **Odak:** Sistemin iç mantığı, mimarisi, deyimler ve akış yolları.
- **Gereksinim:** Kodlama bilgisi ve programatik yapı hakimiyeti gerektirir.
- **Uygulayıcı:** Genellikle programcılar ve mimarlar.
- **Aşamalar:** Tasarım tabanlı ve Kod tabanlı testler.

Kara Kutu Testi



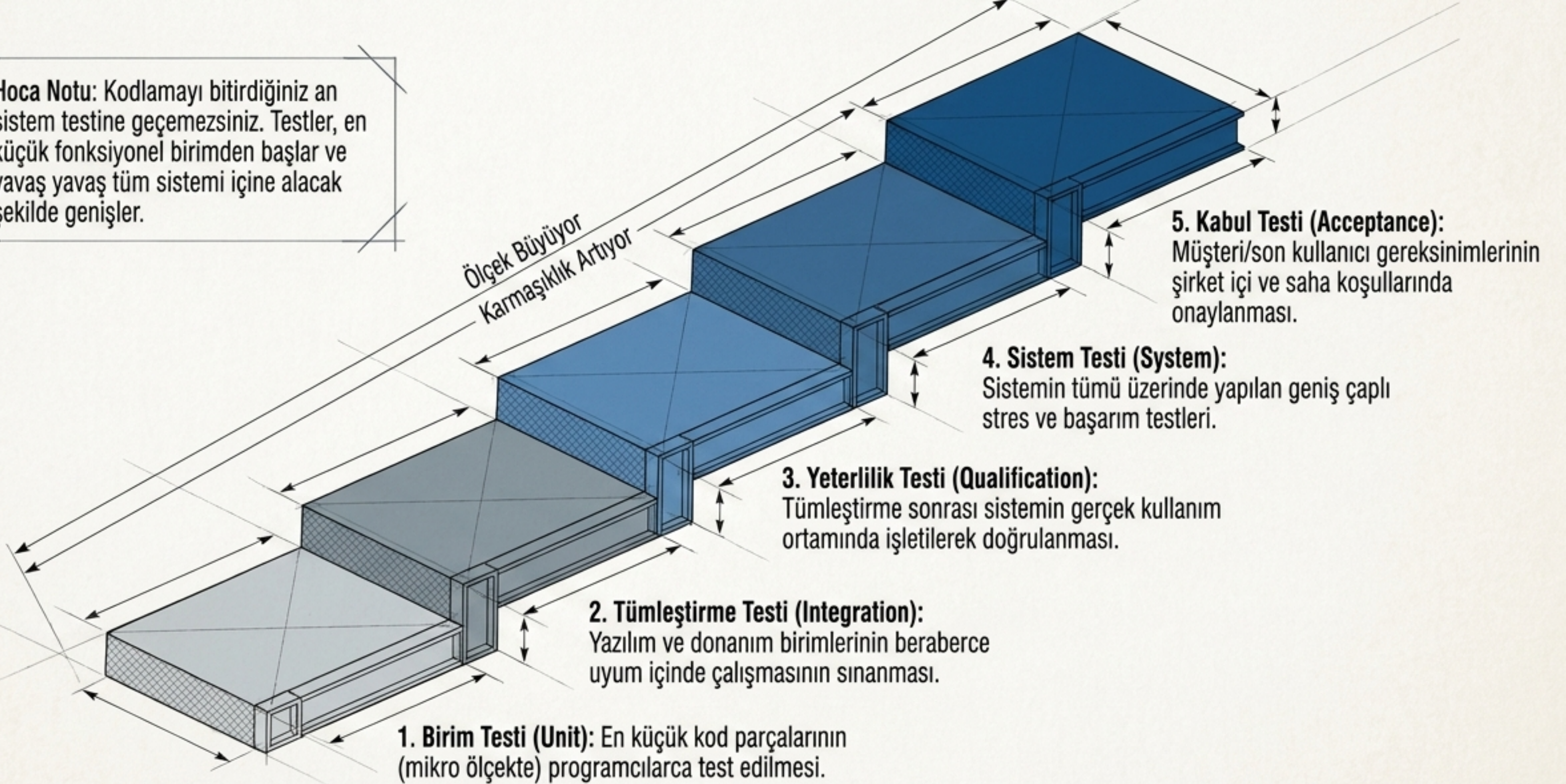
- **Odak:** Kullanıcı arayüzü, girdi-çıkı ilişkileri ve gereksinimlerin karşılanması.
- **Gereksinim:** Kod mimarisi bilgisi gerektirmez; sistemin 'ne yaptığıyla' ilgilenir.
- **Uygulayıcı:** Test uzmanları ve son kullanıcılar.

Hoca Notu Öğrenciler, bu iki konsepti asla unutmayın. Saydam kutu 'Kod doğru yazılmış mı?' diye sorar. Kara kutu ise 'Yazılım doğru çalışıyor mu?' diye sorar. İkisi de şarttır!

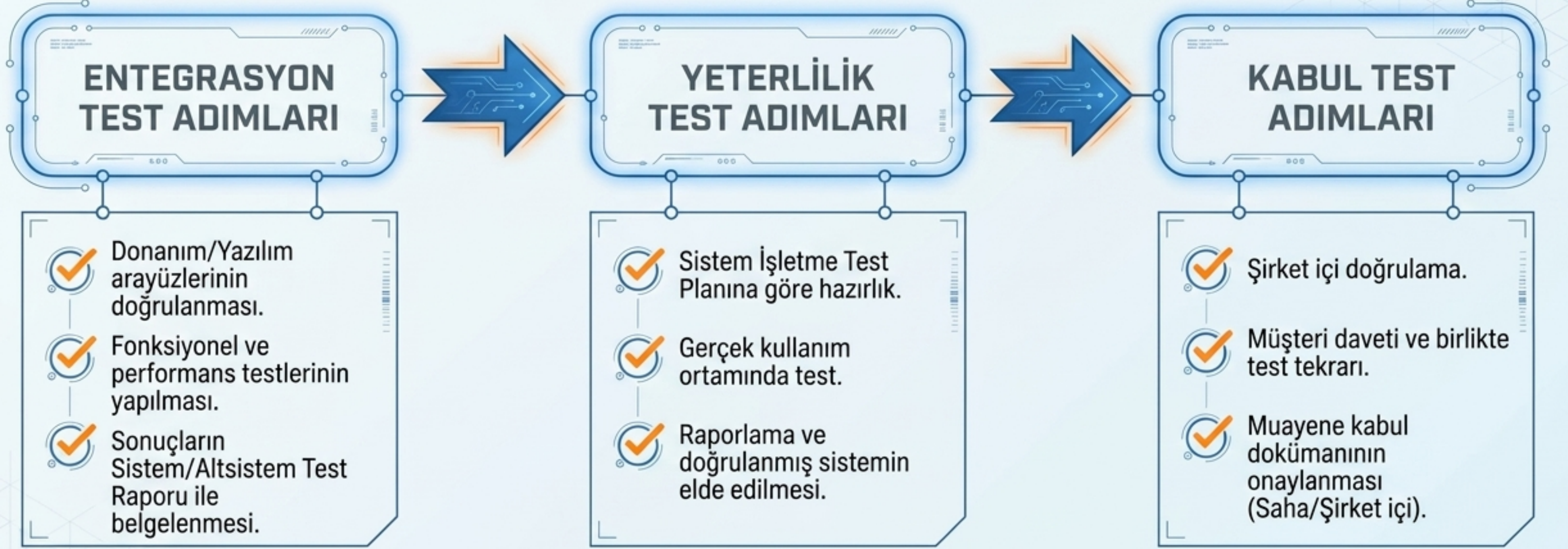
YAZILIM TEST HİYERARŞİSİ



Hoca Notu: Kodlamayı bitirdiğiniz an sistem testine geçemezsiniz. Testler, en küçük fonksiyonel birimden başlar ve yavaş yavaş tüm sistemi içine alacak şekilde genişler.



SÜREÇ YÖNETİMİ: ENTEGRASYON, YETERLİLİK VE KABUL



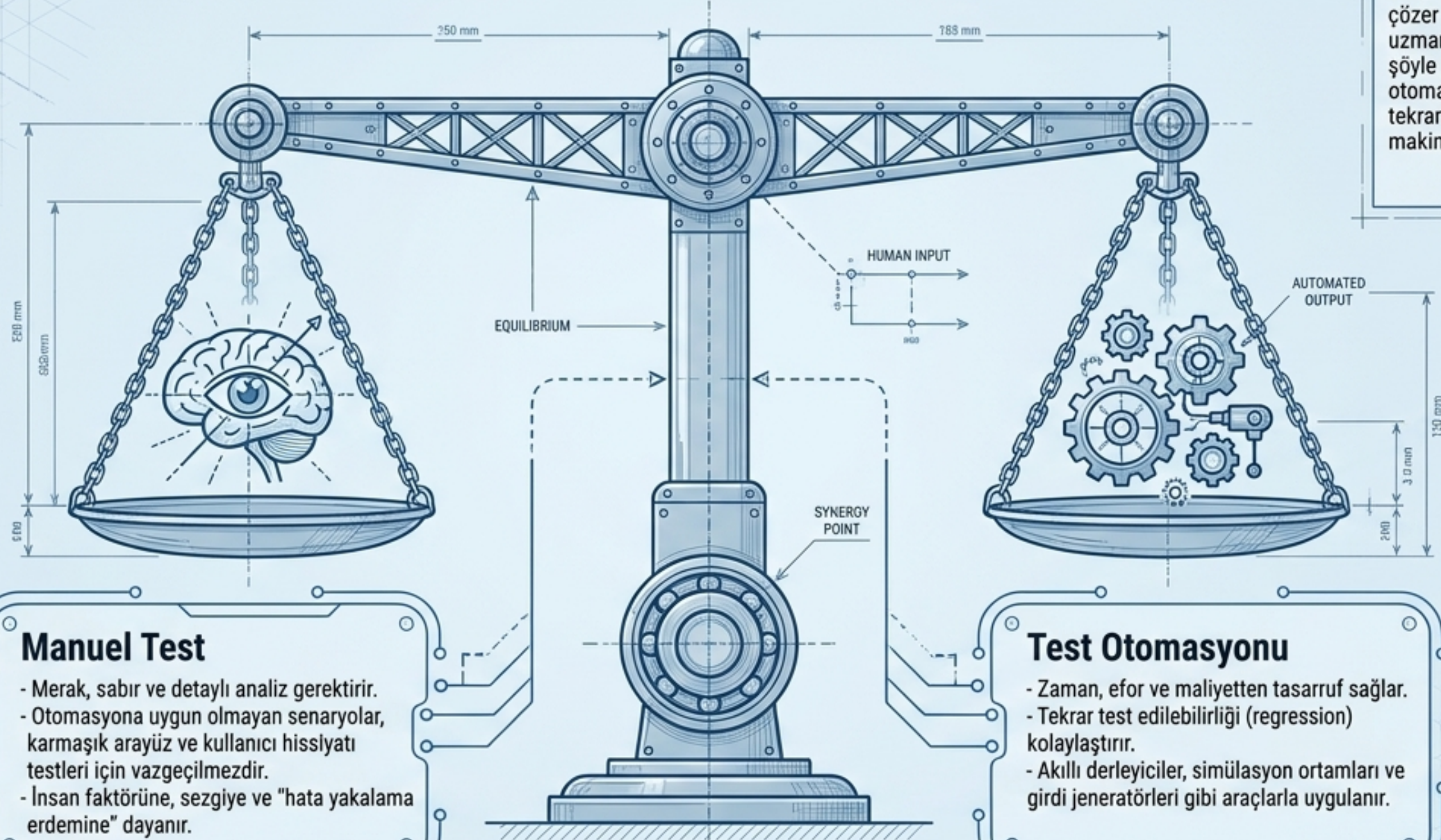
Hoca Notu: Test mühendisliğinde kod yazmak kadar raporlama ve süreç yönetimi de kritiktir. Dokümantasyon olmadan, müşteriye ürünün doğru çalıştığını ispatlayamazsınız.

METODOLOJİ: MANUEL TEST VS. TEST OTOMASYONU



Hoca Notu

Hoca Notu: Sıkça sorulan bir soru: "Otomasyon her şeyi çözer mi?" Hayır. Manuel test uzmanının sezgisi ve "Ya şöyle yaparsam?" yaklaşımı otomatize edilemez. Ancak tekrarlı işleri kesinlikle makinelere bırakmalıyız.



Manuel Test

- Merak, sabır ve detaylı analiz gerektirir.
- Otomasyona uygun olmayan senaryolar, karmaşık arayüz ve kullanıcı hissiyatı testleri için vazgeçilmezdir.
- İnsan faktörüne, sezgiye ve "hata yakalama erdemine" dayanır.

Test Otomasyonu

- Zaman, efor ve maliyetten tasarruf sağlar.
- Tekrar test edilebilirliği (regression) kolaylaştırır.
- Akıllı derleyiciler, simülasyon ortamları ve girdi jeneratörleri gibi araçlarla uygulanır.

KULLANICI VE İŞLEV ODAKLI TESTLER



FONKSİYONEL TEST

Beklenen işlevlerin ve tanımlanmamış / beklenmeyen koşulların (negatif test) kontrolü. Gerçek kullanıcılarda data kaybı riskini engeller.



KULLANICI ARAYÜZÜ (UI) TESTİ

Ekranlar, menüler, butonlar ve yazılımın ön yüzündeki tüm görsel/işlevsel öğelerin pozitif ve negatif adımlarla test edilmesi.



LOKALİZASYON VE DOYUM TESTİ

Desteklenen diller, bölgesel saat/tarih ayarları, kültür odaklı kullanım alışkanlıkları ve nihai müşteri beğenisinin (doyum) sınanması.



Hoca Notu: Bu testler, yazılımınızın dünyayla ve son kullanıcıyla temas ettiği yüzeyidir. Sistemin arkada harika çalışması, kullanıcı arayüzü bozursa veya kendi diline uymuyorsa bir anlam ifade etmez.

ALTYAPI VE SİSTEM ODAKLI TESTLER



PERFORMANS & YÜK TESTİ

Sistemin ağır iş yükü altındaki performansı, dar boğazların (bottleneck) tespiti. Sanal kullanıcı simülasyonları ile gerçek koşul testleri.



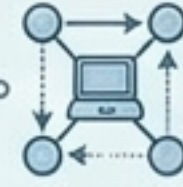
PERFORMANS & YÜK TESTİ

Sistemin ağır iş yükü altındaki performansı, dar boğazların (bottleneck) tespiti. Sanal kullanıcı simülasyonları ile gerçek koşul testleri.



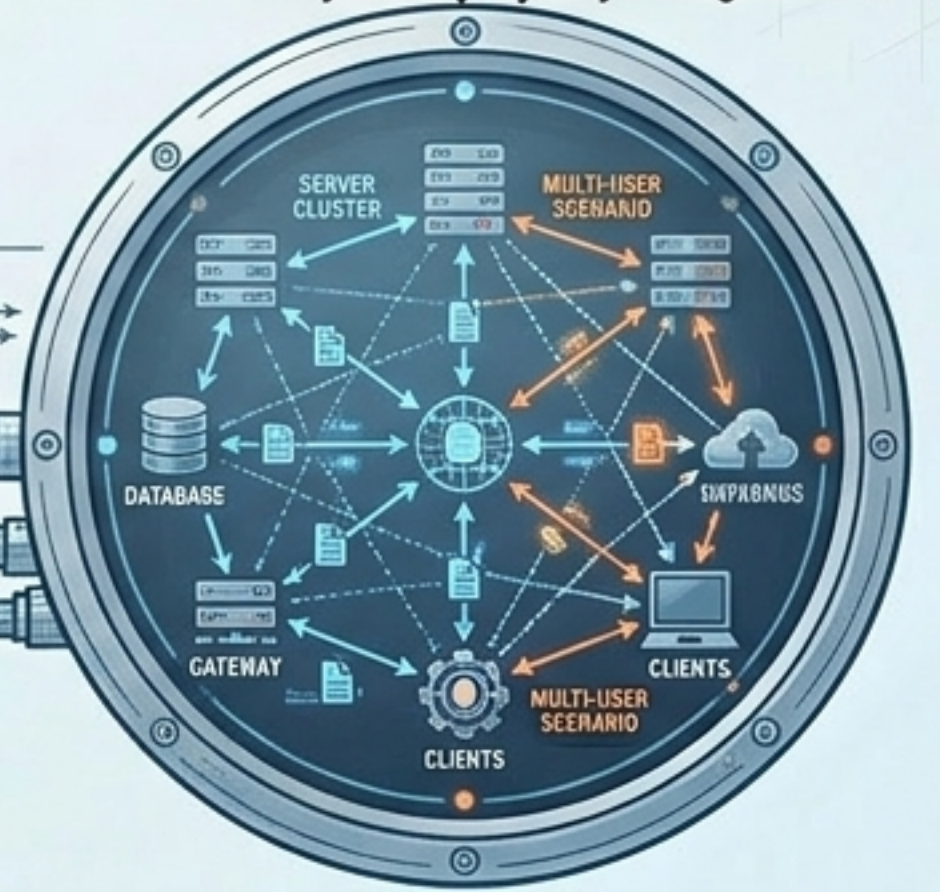
GÜVENLİK TESTİ

İç ve dış kaynaklı yetkisiz erişimlere, kötü amaçlı kullanımlara ve sızmalara karşı sistemin savunma testleri.



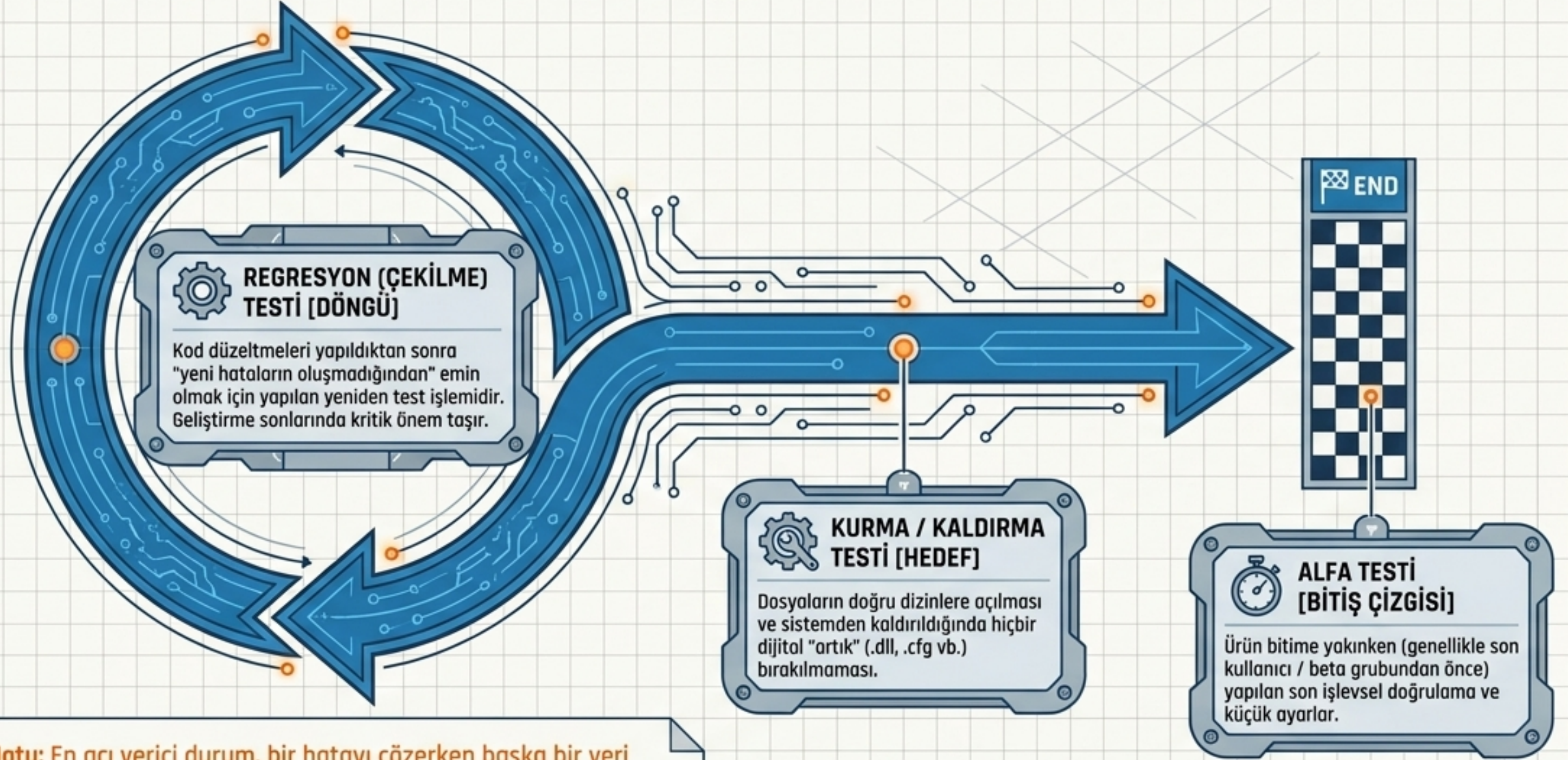
AĞ & UYUMLULUK TESTİ

Farklı donanım, İşletim Sistemi (OS) ve ağ protokolleri altında, çok kullanıcı senaryolarla çalışma yeteneğinin sınanması.



Hoca Notu: Sistemin iskeletini burada test ederiz. Binlerce kullanıcı aynı anda bağlandığında (yük) ya da kötü niyetli biri sisteme sızmaya çalıştığında (güvenlik) kodumuz ayakta kalmalı.

DÖNGÜYÜ KAPATMAK: REGRESYON VE KURULUM TESTLERİ



Hoca Notu: En acı verici durum, bir hatayı çözerken başka bir yeri bozmaktır. Regresyon testleri bizim güvenlik ağıımızdır. Kurulum testleri ise müşterinin yazılımımızla olan ilk ve son izlenimidir.

OTOMATİK TEST CEPHANELİĞİ (ARAÇLAR)



AKILLI DERLEYİCİLER

Hassas tip kontrolü ve yapısal denetim yapar.



DURAĞAN ÇÖZÜMLEYİCİLER

Kaynak kodu yapısal olarak çalıştırılmadan denetler.



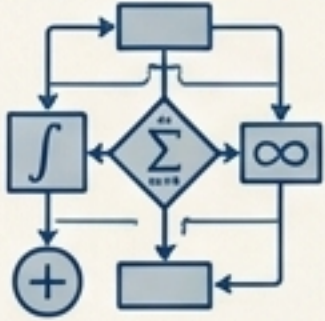
BENZETİM (SİMÜLASYON) ORTAMLARI

Sanal/hedef donanım üstünde test sağlar.



GİRDİ DOSYALARI

Uygulama için devasa test verileri üretir.



SİMGESEL TESTLER

Belirli algoritmaların spesifik sayısal testlerini yapar.



ÇEVRE BENZETİCİLERİ

Özellikle Gömülü ve Gerçek Zamanlı Sistemler için donanımsal çevre şartlarını simüle eder.



Hoca Notu: Özellikle Gömülü, Gerçek Zamanlı ve Büyük Veri Sistemleri kodluyorsak, bu otomasyon araçlarına hakim olmak zorundayız. Mühendis, kendi işini kolaylaştıran aletleri kendi üreten/kullanan kişidir.

BÜYÜK SENTEZ

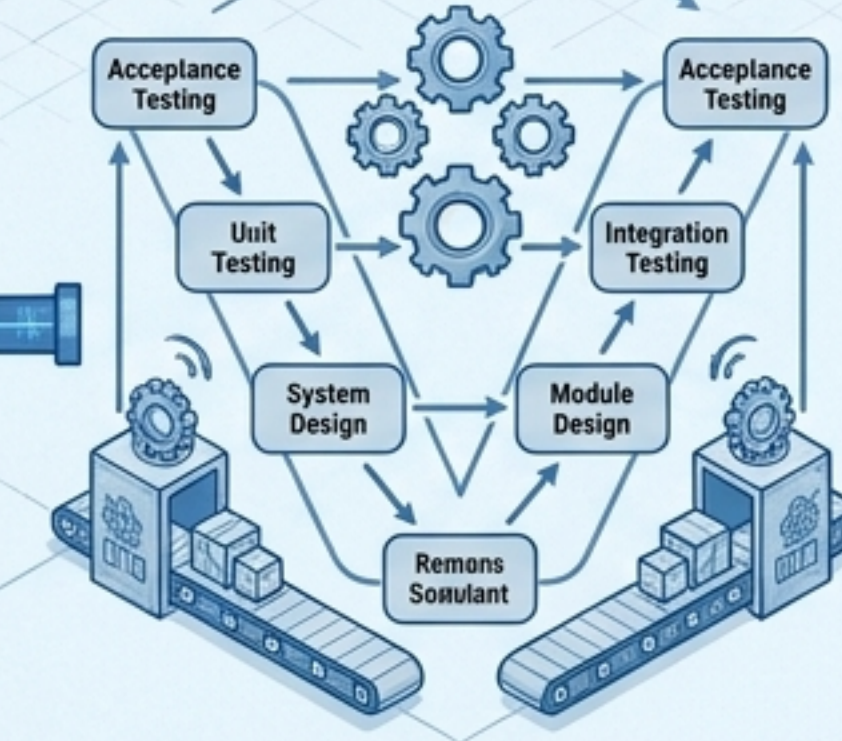


Hoca Notu: Dersimizin özü şudur arkadaşlar: **Kalite test** edilerek **sonradan eklenemez**, ancak sıkı bir **test mimarisiyle baştan tasarlanır**. Kodlarınıza her zaman onları kıracak bir **hacker**, hiç bilmeyen bir kullanıcı ve sistemin sınırlarını zorlayacak bir **mühendis** gibi yaklaşın. Hepinize başarılı projeler diliyorum!

White Box



Gereksinimleri Analiz Et (Saydamlaş)
-> İç mantığı sağlam kur.



Otomasyon ile Güvenceye Al
-> Verimi artır ve regresyonu sağla.



Son Kullanıcıyı Unutma (Kara Kutu)
-> Arayüz ve performansı zorla.

ANAHTAR ÇIKARIM: Yazılım Testi, projenin sonunda yapılan bir "hata arama" eylemi değil, tasarımın ilk gününden ürünün son teslim anına kadar süren kesintisiz bir **KALİTE KÜLTÜRÜDÜR**.